PHASE

12

Storage: Block Devices, ATA Driver and Buffer Cache

Make data survive a reboot: talk to a hard disk, organise devices behind one interface, and cache blocks in memory

Overall progress toward Turk-OS 1.0 (this phase in red) 77 % → 83 %

Duration	2 weeks
Difficulty	★★★★☆ (4/5)
You need	Phase 9: semaphores and mutexes; Phase 4: IRQ handling; Phase 7: the heap and lists
You will have	An ATA PIO driver that identifies, reads and writes the QEMU disk (first polling, then interrupt-driven), a block device layer with partitions from the MBR, a RAM disk from a boot module, a buffer cache with LRU replacement and write-back, and `lsblk` and `hexdump` commands. Data you write survives a reboot, and you can check it from Linux.

1 Why this phase

Everything Turk-OS knows vanishes when QEMU exits. OSTEP's third big idea, **persistence**, starts here: a driver for a disk, and the layers that let the rest of the kernel use disks without knowing how they work. The file system in Phase 13 sits directly on top of this phase and never touches hardware.

You will write the most complete driver so far. The ATA disk is controlled through a handful of I/O ports, it is slow compared with the CPU, and it signals completion with an interrupt, so the task that asked for data should sleep until it arrives. That brings together port I/O (Phase 2), interrupts (Phase 4) and blocking synchronisation (Phase 9). Above the driver you will build the two layers that every UNIX kernel has in some form: a device-independent **block layer** and a **buffer cache** that keeps recently used blocks in memory, because reading from RAM is thousands of times faster than reading from a disk.

2 Concepts you need

2.1 How a disk is organised

A disk stores data in fixed-size **sectors** of 512 bytes. Old interfaces addressed them by cylinder, head and sector (CHS); everything modern uses **logical block addressing** (LBA), which simply numbers the sectors 0, 1, 2, … OSTEP chapter 37 builds a model of a mechanical disk's access time: **seek** time to move the arm, **rotational** delay waiting for the sector to come round, and **transfer** time. The model explains why sequential access is so much faster than random access, why disk schedulers reorder requests (FIFO, SSTF, SCAN, C-SCAN), and why every file system tries to keep related blocks close together. QEMU's disk is a file on your host, so none of the timing is real, but the interface is the same as on a real PC.

2.2 The layers of I/O software

MOS §5.3 divides I/O software into four layers, and Turk-OS follows it (Figure 1). The **interrupt handler** does the minimum and wakes whoever is waiting. The **device driver** knows one kind of hardware. The **device-independent** layer (block devices, partitions, the buffer cache) gives every

driver the same interface and does the work common to all of them. **User-level** software (the file system, then programs) only ever sees that interface.

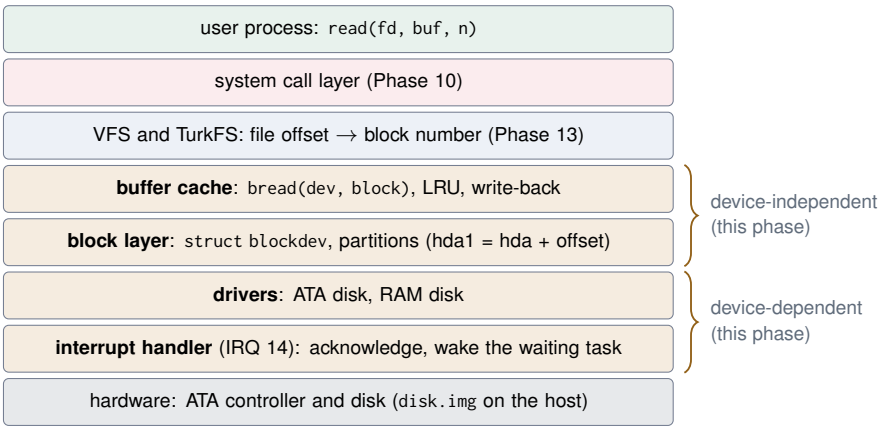


Figure 1: The storage stack. A `read` on a file passes through every layer; a cache hit stops at the buffer cache.

2.3 Programmed I/O on the ATA interface

The primary ATA channel of a PC is a set of eight I/O ports starting at `0x1F0`, plus a control port at `0x3F6` (Table 1). To read, the driver selects the drive, writes the sector count and the 28-bit LBA into four registers, and writes the `READ SECTORS` command. When the drive has a sector ready it sets **DRQ** in the status register (and raises IRQ 14 if interrupts are enabled), and the driver reads 256 16-bit words from the data port. That is **programmed I/O** (PIO): the CPU copies every word. **DMA**, where the controller copies directly into memory, is faster and is a stretch goal (OSTEP §36.5, MOS §5.1.4).

Port	Read	Write
`0x1F0`	data (16-bit)	data (16-bit)
`0x1F1`	error	features
`0x1F2`	sector count	sector count
`0x1F3`, `0x1F4`, `0x1F5`	LBA bits 0–7, 8–15, 16–23	same
`0x1F6`	drive/head	`0xE0` | (slave $\ll$ 4) | LBA bits 24–27
`0x1F7`	status: BSY `0x80`, DRDY `0x40`, DF `0x20`, DRQ `0x08`, ERR `0x01`	command: `0xEC` identify, `0x20` read, `0x30` write, `0xE7` flush
`0x3F6`	alternate status (reading it doesn't acknowledge an interrupt)	device control: bit 1 nIEN disables IRQs

Table 1: The primary ATA channel's registers in 28-bit LBA mode.

2.4 The buffer cache

The file system reads the same blocks over and over: the superblock, the bitmaps, the inodes of directories on every path. The **buffer cache** keeps a fixed pool of block-sized buffers in memory. `bread(dev, block)` returns the buffer for that block, reading it from disk only if it isn't cached; the caller holds the buffer's lock while using it and gives it back with `brelse`. When all buffers are in use, the least recently used one that nobody holds is reused. Writes can go to disk immediately (**write-through**) or be delayed until the buffer is evicted or `sync` runs (**write-back**), which is faster but loses data in a crash: the central trade-off of Phase 13's crash-consistency discussion. The cache also guarantees there is only ever *one* buffer per block, so two tasks can't hold diverging copies.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	OSTEP	Ch. 36 *I/O Devices*, especially §36.8 *A Simple IDE Disk Driver*	405–416	Section 36.8 is essentially the driver of Tasks 12.2–12.4.
REQUIRED	OSTEP	Ch. 37 *Hard Disk Drives*	419–436	The disk model, access time, and disk scheduling.
REQUIRED	MOS	§5.2–5.3 *Principles of I/O Software*, *I/O Software Layers*	351–369	The layering this phase implements.
REQUIRED	MOS	§5.4 *Disks* (hardware, formatting, arm scheduling, error handling, stable storage)	369–388	What a disk driver has to deal with in practice.
RECOMMENDED	OSDI	§3.4–3.7: I/O in MINIX 3, block devices, RAM disks, disks	252–302	A complete disk driver and RAM disk with explanations.
RECOMMENDED	OSC	§11.1–11.2 *Mass-Storage Structure*, *HDD Scheduling*; §11.5 *Storage Device Management*	449–461, 463–467	HDDs versus SSDs, scheduling, partitions and boot blocks.
RECOMMENDED	OSC	§12.3–12.5 *Application I/O Interface*, *Kernel I/O Subsystem*, *Transforming I/O Requests*	500–519	Block versus character devices; buffering and caching in the kernel.
OPTIONAL	OSTEP	Ch. 38 *RAID*	437–456	Combining disks for speed and reliability.
OPTIONAL	OSDI	§5.6.5 *The Block Cache*	557–559	The MINIX buffer cache, which you will meet again in Phase 13.
RECOMMENDED	LKD	Ch. 14, *The Block I/O Layer*	289–304	Buffers, `bio`, request queues and I/O schedulers.
RECOMMENDED	LKD	Ch. 8, *Work Queues*	149–157	Running disk completion work outside the interrupt handler.
OPTIONAL	LKD	Ch. 16, *The Page Cache and Page Writeback*	323–336	Why Linux folded its buffer cache into the page cache (Going further).

The OSDev Wiki page *ATA PIO Mode* is the practical reference for the register sequences, including the quirks of real hardware that QEMU forgives.

Real-system lens: how MINIX 3 does it

The MINIX hard disk driver, `drivers/at_wini/at_wini.c (12100)`, is a user-space process that receives read and write requests as messages. Its `w_identify` does what your Task 12.2 does, and `w_transfer` with `com_out` issues commands and waits for the interrupt. All block drivers share the main loop in `drivers/libdriver/driver.c (11000)`: that is the device-independent layer, done as a library. The RAM disk is `drivers/memory/memory.c (11600)`. The block cache is part of the file-system server, in `servers/fs/cache.c (22400)`, with `get_block` and `put_block` playing the roles of your `bread` and `brelse`.

Linux, for comparison. LKD Linux describes I/O with `bio` structures in request queues that an I/O scheduler reorders, completes requests in bottom halves, and long ago folded its buffer cache into the unified **page cache**. Turk-OS keeps a buffer cache because TurkFS works in blocks; the page cache is a Going-further idea.

4 Build it

Task 12.1 A disk for QEMU

Create a raw disk image and attach it as the primary master; add a Makefile target so every run has it. Booting with `-kernel` or from the CD image doesn't conflict, because QEMU attaches the CD to the secondary channel.

terminal

```
qemu-img create -f raw build/disk.img 64M
qemu-system-i386 -kernel build/kernel.elf -serial stdio \
    -drive file=build/disk.img,format=raw,if=ide,index=0,media=disk
```

Verify: The QEMU monitor's `info block` lists `ide0-hd0` backed by `build/disk.img`.

Task 12.2 Find the drive with IDENTIFY

Select the master drive (`0xA0` to `0x1F6`), zero the count and LBA registers, send `0xEC`, and read the status. Zero means no drive. Otherwise wait for BSY to clear; if the LBA mid and high registers are now non-zero, the device is not an ATA disk. Wait for DRQ (or ERR), then read 256 words. Words 27–46 hold the model name, with the two bytes of every word swapped, and words 60–61 hold the number of 28-bit addressable sectors.

Verify: At boot the kernel prints `hda: QEMU HARDDISK, 131072 sectors (64 MiB).`

Task 12.3 Read and write sectors, polling

Write `ata_read` and `ata_write` for up to 255 sectors per call. After selecting the drive, wait about 400 ns by reading the alternate status port four times, because the status is not valid immediately. Before every sector, wait until BSY is clear and DRQ is set, and check ERR and DF. Move data with `rep insw` and `rep outsw`. After writing, send `CACHE FLUSH` (`0xE7`) and wait for BSY to clear, or the data may still be sitting in the drive's cache.

kernel/drivers/ata.c

```
#define ATA_DATA 0x1F0
#define ATA_SECCOUNT 0x1F2
#define ATA_LBA0 0x1F3
#define ATA_LBA1 0x1F4
#define ATA_LBA2 0x1F5
#define ATA_DRIVE 0x1F6
#define ATA_CMD 0x1F7
#define ATA_STATUS 0x1F7
#define ATA_ALTSTATUS 0x3F6
#define ST_BSY 0x80
#define ST_DF 0x20
#define ST_DRQ 0x08
#define ST_ERR 0x01

static inline void insw(uint16_t port, void *buf, uint32_t words)
{
    __asm__ volatile ("rep insw" : "+D"(buf), "+c"(words) : "d"(port) : "memory");
}

static void delay400ns(void) { for (int i = 0; i < 4; i++) inb(ATA_ALTSTATUS); }

static int wait_ready(void)
{
    uint8_t s;
    while ((s = inb(ATA_STATUS)) & ST_BSY)
```

```
        ;
    if (s & (ST_ERR | ST_DF))
        return -EIO;
    return (s & ST_DRQ) ? 0 : -EIO;
}

int ata_read(uint32_t lba, uint8_t count, void *buf)
{
    outb(ATA_DRIVE, 0xE0 | ((lba >> 24) & 0x0F));   /* LBA mode, master, LBA 24..27 */
    delay400ns();
    outb(ATA_SECCOUNT, count);
    outb(ATA_LBA0, lba & 0xFF);
    outb(ATA_LBA1, (lba >> 8) & 0xFF);
    outb(ATA_LBA2, (lba >> 16) & 0xFF);
    outb(ATA_CMD, 0x20);                            /* READ SECTORS */
    for (uint8_t i = 0; i < count; i++) {
        if (wait_ready() < 0)
            return -EIO;
        insw(ATA_DATA, (uint8_t *)buf + i * 512, 256);
    }
    return 0;
}
```

Verify: Write a pattern to sectors 100–107, read it back and compare. Then exit QEMU and look at the image on the host with `hexdump -C -s $((100*512)) -n 64 build/disk.img`: your pattern is there.

Task 12.4 Let the disk interrupt

Polling wastes the CPU while the disk works. Enable disk interrupts (clear nIEN in the device control register), unmask IRQ 14 on the slave PIC and IRQ 2 on the master, and register a handler for vector 46 that reads the status register (which acknowledges the interrupt) and calls `sem_up`. In `ata_read`, replace the busy wait before each sector with `sem_down`. Serialise requests with a mutex, since the controller can only do one thing at a time. This handler already follows the top-half rule of Task 4.7: it only acknowledges the device and wakes the waiting task, and the slow part, copying 256 words from the data port, runs in that task's context, where it may be preempted. Linux calls this pattern a completion (LKD ch. 10, *Completion Variables*, p. 197).

Verify: While one task reads 10 MiB from the disk, another task that prints once a second keeps its rhythm, and `ps` shows the reader blocked most of the time.

Task 12.5 A block device layer

Define `struct blockdev` with a name, the sector size, the number of sectors, `read` and `write` function pointers, and a private pointer for the driver. Keep a registry, and add `blockdev_find("hda")`. From now on, nothing above this layer calls `ata_read` directly.

include/blockdev.h

```
struct blockdev {
    char     name[8];                   /* "hda", "hda1", "ram0" */
    uint32_t sector_size;               /* 512                   */
    uint32_t nsectors;
    int    (*read)(struct blockdev *d, uint32_t lba, uint32_t count, void *buf);
    int    (*write)(struct blockdev *d, uint32_t lba, uint32_t count, const void *buf);
    void    *priv;                      /* driver state; for a partition: parent + offset */
};
```

Task 12.6 Partitions

Read sector 0 of each disk. If bytes 510–511 are `55 AA`, parse the four 16-byte partition entries at offset `0x1BE`: byte 4 is the type, bytes 8–11 the first LBA, bytes 12–15 the number of sectors. Register each used entry as a block device (`hda1`…`hda4`) whose read and write add the offset and reject any request that would cross the partition's end. Partition the image on the host; type `81` is the traditional MINIX type, which fits the file system of Phase 13.

terminal

```
echo 'start=2048, type=81' | sfdisk build/disk.img
```

Verify: `lsblk` lists `hda` (64 MiB) and `hda1` starting at sector 2048, and a read beyond the end of `hda1` fails with `-EINVAL`.

Task 12.7 A RAM disk

Register any boot module named `ramdisk` as the block device `ram0`; its read and write functions are just `memcpy`. It is useful for testing the file system without the ATA driver in the way, and in Phase 13 the initial root file system can live there.

Task 12.8 The buffer cache

Write `bread`, `bwrite`, `bdirty`, `brelse` and `bsync` over a pool of 64 buffers of 1 KiB (the block size of the Phase 13 file system, two sectors). Each buffer has a device, a block number, valid and dirty flags, a reference count, a mutex held by whoever uses the data, and a position in an LRU list; add a small hash table from (device, block) to buffer once the linear search shows up in profiles. Look up and claim buffers under one spinlock, so two tasks asking for the same block get the same buffer.

include/bcache.h

```
#define BLOCK_SIZE 1024

struct buf {
    struct blockdev *dev;
    uint32_t         blockno;
    bool             valid;          /* data holds the block's contents    */
    bool             dirty;          /* data must be written back          */
    int              refcnt;         /* users holding it (not reusable)    */
    struct mutex     lock;           /* held while a task uses data        */
    struct list_node lru;            /* most recently released at the head */
    uint8_t          data[BLOCK_SIZE];
};

struct buf *bread(struct blockdev *dev, uint32_t blockno);   /* returns locked, valid */
void        bdirty(struct buf *b);                           /* write back later      */
void        bwrite(struct buf *b);                           /* write now             */
void        brelse(struct buf *b);                           /* unlock, move to MRU   */
void        bsync(void);                                     /* flush every dirty one */
```

Verify: Reading the same block twice causes one disk read; a `cache` command prints hits, misses and dirty buffers; after `bsync` the dirty count is zero.

Task 12.9 Tools and tests

Add `lsblk` and `hexdump <dev> <block>` to the monitor. Add tests for block device reads and writes on `ram0`, partition bounds checking, cache hits and eviction (touch 100 different blocks with 64 buffers), and a persistence test: write a known block, `bsync`, reboot with the same image, and check it.

5 Testing and debugging

Symptom	Likely cause	What to do
IDENTIFY returns only zeros or hangs	Wrong drive select, or no disk attached at `index=0`	Check `info block`; select with `0xA0` and wait 400 ns before reading status.
Reads return the previous sector's data	DRQ not checked before each sector of a multi-sector read	Call `wait_ready` before every 256-word transfer.
Written data is missing after QEMU exits	No `CACHE FLUSH` after writes, or dirty buffers never synced	Send `0xE7` after writing; call `bsync` before shutdown.
IRQ 14 never arrives	IRQ 2 on the master or IRQ 14 on the slave still masked, or nIEN set	Unmask both and clear nIEN in `0x3F6`; `info pic` shows the masks.
Two tasks see different contents for the same block	Two buffers were created for the same block	Do lookup and allocation atomically under the cache's spinlock.
Partition data lands in the wrong place	The partition offset is applied twice, or not at all	Test with a known pattern at the partition's first sector and check with `hexdump -C` on the host.

6 Milestone

Milestone: Data survives

- ☐ The ATA driver identifies the QEMU disk, and reads and writes with interrupts and a blocking wait.
- ☐ All disk access goes through `struct blockdev`; partitions from the MBR appear as their own devices.
- ☐ The RAM disk works from a boot module.
- ☐ The buffer cache serves repeated reads from memory and writes dirty blocks back on `bsync`.
- ☐ Data written by Turk-OS survives a reboot and is visible with `hexdump -C` on the host.

Self-check questions

1. What are the three components of disk access time, and which one does sequential access avoid?
2. Compare programmed I/O and DMA. What does the CPU do during each?
3. Why must the driver check DRQ before every sector rather than once per command?
4. What does the device-independent block layer buy you? Give two concrete examples in Turk-OS.
5. What does the buffer cache guarantee besides speed? What is the risk of write-back caching?
6. Describe the layout of an MBR and one partition entry.

7 Going further

Enumerate the PCI bus through configuration ports `0xCF8/0xCFC`, find the IDE controller, and use its bus-master DMA engine so the CPU no longer copies every word. Queue requests and sort them with C-SCAN (OSTEP §37.5, MOS §5.4.3). Add read-ahead: when block n is read, start reading $n+1$ in the background. Or write a `virtio-blk` driver, which is how modern virtual machines expose disks, and compare its speed with ATA.

Next: Phase 13 File Systems: VFS, initrd and TurkFS →

Books used in this plan

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at `https://littleosbook.github.io`.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. **LOB** and **OSTEP** are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.